

```
m says hello and asks for m  
  
world!')  
s your name?') # ask for th  
t()  
  
good to meet you, ' + myNam  
ngth of your name is:')  
ame))  
s your age?') # ask for the  
()  
ll be ' + str(int(myAge) +
```

Python Main Function and Command Line Arguments

Welcome to the world of Python programming! In this presentation, we'll explore the main function in Python and the significance of command line arguments in Python programs.



by Prof. Mohd Iqbal Bhat



Main function in Python

- The main function is the entry point of a Python program. It allows us to organize our code and perform specific actions when the program starts.
- In Python, the main function is a convention rather than a built-in feature.
- It's a common practice to define a main function in Python scripts, especially in larger programs.
- The main function serves as the entry point for your script, where execution begins.



Benefits of Using a Main Function

- **Code Modularity:** The main function allows you to separate different parts of your code, making it easier to maintain and understand.
- **Reusability:** You can easily reuse functions and classes defined in your script by importing them in other scripts.
- **Testing and Debugging:** Testing and debugging are more manageable when the main logic is contained within a single function.

How to Define a Main Function

```
def main():  
    """Prints a greeting message."""  
    print("Hello, world!")  
  
if __name__ == '__main__':  
    main()
```

- `def main()`: defines the main function, where you place your program's logic.
- `if __name__ == "__main__"`: checks whether the script is being run as the main program or imported as a module. If it's the main program, it calls the `main()` function.

Command Line Arguments

- Command line arguments are a way to pass information to Python scripts when they are run from the command line or terminal.
- They are essential for creating scripts that are versatile and can accept input from users or other programs.
- Command line arguments are values or parameters provided to a script when it's executed from the command line.
- They are used to configure or customize the behavior of a Python script.

How to Access Command Line Arguments:

`sys.argv` list

- Python provides the `sys.argv` list, part of the `sys` module, to access command line arguments.
- `sys.argv` is a list that includes the name of the script being run as its first element, followed
- by any arguments provided by the user.
- To access command line arguments, you can simply index `sys.argv`.
- The first element (index 0) is the script name, and any additional elements are the arguments.
- For example, if you run `python myscript.py arg1 arg2`,
then `sys.argv[0]` is "myscript.py," `sys.argv[1]` is "arg1," and `sys.argv[2]` is "arg2."

How to Access Command Line Arguments:

sys.argv list

```
import sys

def main():
    """Prints the command line arguments."""
    print("Name of the Program is:",sys.argv[0])
    for arg in sys.argv[1:]:
        print(arg)

if __name__ == '__main__':
    main()
```

Usage of Command Line Arguments:

```
import sys

def main():
    """Prints the contents of a file."""
    if len(sys.argv) != 2:
        print("Usage: python3 print_file.py <filename>")
        return

    filename = sys.argv[1]

    with open(filename, "r") as f:
        for line in f:
            print(line)

if __name__ == '__main__':
    main()
```


Conclusion

Understanding the main function and command line arguments in Python can greatly enhance the functionality and flexibility of your programs. Start exploring this powerful feature today!